

M1

Méthode – transversal

Faire écrire une requête par un LLM

Ce qu'il faut mettre dans le prompt pour obtenir du SQL juste – et ce que la machine ne pourra jamais deviner à votre place.

TU ES ICI SI...

- ▶ « L'IA m'a proposé une requête qui utilise une colonne qui n'existe pas. »
- ▶ « La requête compile, mais elle ne répond pas à l'énoncé. »
- ▶ « Je ne sais pas quoi donner comme contexte : le schéma entier ? un extrait ? »
- ▶ « Je repose la même question et j'obtiens une requête différente. Laquelle est la bonne ? »

Un LLM ne consulte pas votre base : il **prédit la suite la plus probable de votre prompt**. Tout ce que vous n'y avez pas mis sera soit inventé, soit moyenné sur les milliers de bases vues à l'entraînement. Sur un même modèle, passer d'une description en prose à un schéma structuré change les résultats davantage que la plupart des réglages. **Attention** : les LLM sont des outils très énergivores, et leur consommation est très dépendante du nombre de tokens en entrée (taille du prompt). Veillez à rester raisonnables au niveau de vos usages.

1. Ce que le prompt doit contenir

Élément	Pourquoi il change le résultat
Un schéma structuré	Des instructions <code>CREATE TABLE</code> (ou une liste <code>table(col1, col2, ...)</code>) font nettement mieux qu'une description en prose. Les deux formats structurés se valent : c'est <i>la structure</i> qui compte. Les tables hors sujet distraient le modèle. Sélectionnez avant de coller.
Les clés	Primaires et surtout étrangères . Les retirer dégrade fortement la précision : sans elles, le modèle devine le chemin de jointure – et le devine mal dès qu'il y a une table-association.
Quelques lignes	Trois suffisent. Elles montrent le <i>format réel</i> des valeurs : la catégorie s'appelle 'Horreur', pas 'horror'. En donner beaucoup n'améliore rien et coûte du contexte.
La connaissance métier	Ce que le schéma ne dit pas : ici, <code>return_date</code> à NULL signifie « location en cours », pas « pas de location ». Aucun modèle ne peut l'inventer.

Un prompt qui contient les quatre éléments – et qui impose déjà une démarche

Contexte. PostgreSQL 16. Voici l'extrait utile du schéma :

```
CREATE TABLE category (category_id int PRIMARY KEY, name text);
CREATE TABLE film (film_id int PRIMARY KEY, title text, length int,
                    category_id int REFERENCES category(category_id));
CREATE TABLE customer (customer_id int PRIMARY KEY, last_name text, city text);
CREATE TABLE rental (rental_id int PRIMARY KEY,
                      film_id int REFERENCES film(film_id),
                      customer_id int REFERENCES customer(customer_id),
                      return_date date);
```

Précisions. `category.name` vaut 'Action', 'Documentaire', 'Animation' ou 'Horreur'. Trois lignes de film : (1, 'Academy Dinosaur', 86, 4), (5, 'Escape Metropolis', 141, 1), (9, 'Island Empire', NULL, 3). Une durée à **NULL** est une durée inconnue ; une `return_date` à **NULL** est une location *en cours*, et non l'absence de location.

Question. Pour chaque client, y compris ceux qui n'ont jamais rien loué, le nombre de ses locations en cours. Une ligne du résultat = un client.

Consignes. Une seule requête, SQL standard, sans vue ni fonction. Commence par dire ce que représente une ligne du résultat, quelles tables sont nécessaires et par quelles clés elles se joignent ; n'écris la requête qu'ensuite.

LES TROIS OUBLIS LES PLUS COÛTEUX

Pas de clés étrangères → jointure inventée ou approximée. **Pas de valeurs d'exemple** → `WHERE name = 'Horror'` sur une base en français. **Tout le schéma collé d'un bloc** → le modèle s'accroche à une colonne homonyme sans rapport avec la question.

2. Guider le raisonnement plutôt que demander la réponse

Demander directement le SQL revient à parier sur un coup unique : les travaux du domaine font mieux en imposant une *démarche*.

- ▶ **Décomposer** – demander d'abord un *plan en français* : « une ligne du résultat, c'est quoi ? quelles tables faut-il ? par quelles clés se joignent-elles ? », et la requête seulement ensuite. Un plan faux se repère en trois lignes ; une requête fautive, beaucoup plus difficilement.
- ▶ **Faire expliciter les étapes** (« procède clause par clause ») – le raisonnement en chaîne aide sur les requêtes complexes, et il vous **donne prise** : vous voyez à quelle clause le modèle a dérapé, au lieu de constater seulement que le résultat est faux.
- ▶ **Fixer le cadre** – SGBD et version, une seule requête, pas de vue ni de fonction. Sans cela, vous récupérerez du dialecte MySQL une fois sur trois.

3. Résumé

AVANT D'ENVOYER, AVANT DE COPIER

Avant d'envoyer – à condition d'avoir le droit de communiquer au LLM les informations nécessaires, mon prompt contient-il le schéma *structuré et réduit*, les **clés étrangères**, quelques valeurs réelles, le sens des **NULL** et la granularité attendue (« une ligne = un client ») ?

Avant de copier – suis-je capable de **justifier chaque clause** ? Cette étape n'est pas automatisable : une requête est correcte vis-à-vis d'un énoncé et d'un schéma, pas vis-à-vis d'un résultat plausible.

Ce qu'aucun prompt ne réparera – le modèle n'a ni votre énoncé complet, ni le contexte métier, ni les cas limites de vos données. Un bon prompt *déplace* le travail de spécification et de vérification ; il ne le supprime pas, et c'est ce travail-là qui est évalué.

VÉRIFIE QUE TU AS COMPRIS

1. Vous décrivez les quatre tables en prose, sans mentionner les clés. Quel type d'erreur devient le plus probable ?
2. Le modèle répond `WHERE c.name = 'Horror'`. Quel élément manquait dans le prompt ?
3. Trois fois la même question, trois fois la même requête. Qu'en conclure sur sa justesse ?
4. Vous voulez « le nombre de locations de chaque client, y compris ceux qui n'en ont aucune ». Que précisez-vous dans le prompt pour éviter le `COUNT(*)` à 1 ?

RÉPONSES

1. Une jointure fautive ou inventée : sans clés étrangères, le chemin entre `customer` et `category` est deviné et les tables-passerelles sautent. **2. Les lignes d'exemple** : rien dans un `CREATE TABLE` ne dit que le catalogue est en français. **3. Rien.** La répétition réduit la variance, pas le biais : les trois réponses peuvent partager la même mauvaise lecture de l'énoncé. Seule la relecture tranche. **4. Préciser la granularité et le cas vide** : « une ligne par client, y compris les clients sans aucune location, qui doivent afficher 0 ». Et vérifier ensuite que la réponse utilise bien une jointure externe et `COUNT(r.rental_id)` (cf CM 1).

D'où viennent ces recommandations – cette fiche condense la section 4 (« Prompt Engineering », §4.1 à §4.3) de L. Shi, Z. Tang, N. Zhang, X. Zhang et Z. Yang, *A Survey on Employing Large Language Models for Text-to-SQL Tasks*, *ACM Computing Surveys* 58(2), art. 54, 2025 – doi.org/10.1145/3737873